Paper Deep Research

Shitian Yang
Department of Computer Science
Johns Hopkins University
syang214@jh.edu

Ruijie Tao
Department of Computer Science
Johns Hopkins University
rtao12@jh.edu

Jian Huang
Department of Computer Science
Johns Hopkins University
jhuan243@jh.edu

Abstract

The abstract paragraph should be indented ½ inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

1 Introduction

Modern research fields evolve at a pace that increasingly outstrips any individual's reading capacity. Even for a narrowly scoped idea, identifying the most relevant prior work, checking whether key claims are actually supported by primary evidence, and understanding how techniques emerged and diverged over time can require scanning dozens to hundreds of papers. Each paper may contain nontrivial experimental details, implicit assumptions, and critical implementation choices scattered across text, tables, and figures. As a result, the bottleneck is no longer only retrieving papers, but turning a growing corpus into actionable understanding that remains grounded in traceable evidence.

Recent open-source "deep research agents" show that LLM-based systems can autonomously search the web and synthesize long-form reports with citations. For example, STORM aims to produce Wikipedia-style articles with structured organization and supporting references [20], while GPT-Researcher represents a widely adopted engineering pattern that automatically collects sources and writes citation-backed research reports [4]. These systems can significantly reduce the time-to-overview and help users rapidly orient within an unfamiliar topic.

However, most existing agents still emphasize a search-and-summarize workflow: they generate fluent narratives, but provide limited diagnostic analysis. In practice, researchers often need to know which technical facets are missing, where evidence is weak or indirect, which statements lack traceable support, whether code and experimental settings are sufficiently reproducible, and how specific design choices emerged historically across lineages of work. Existing agents often help answer "what is known?" but only partially support "what is missing, uncertain, or non-reproducible?" This gap becomes more consequential as agents are used for higher-stakes steps beyond writing. Agent Laboratory, for instance, explicitly connects literature review to experimentation and report writing, illustrating a direction where agents participate in the full research lifecycle rather than only drafting text [18]. As agents propose conclusions, recommend citations, or even run experiments, researchers must be able to inspect where each claim comes from, what sources support it, and what gaps remain.

The need for auditable, evidence-traceable literature understanding builds on a long history of studying scientific progress through citation networks. Citation indexing provided an early foundation

Preprint.

by treating citations as explicit links between ideas, enabling navigation of the literature as a network rather than a pile of isolated documents [6]. Classic bibliometric primitives then operationalized relatedness and structure using reference lists alone: bibliographic coupling connects papers that cite similar prior work [11], while co-citation links papers that are frequently cited together by later work [22]. As datasets and computational tools improved, these ideas broadened into system-level views of science, including influence ranking, clustering, and science mapping, where citation networks serve as observable traces of knowledge diffusion and specialization [21]. Temporal techniques such as burst detection further support identifying abrupt increases in attention that often correspond to emerging topics or turning points [12]. In this sense, modern technology-evolution pipelines can be viewed as an automation layer over a decades-old objective: reconstructing how ideas branch, converge, and gain influence over time.

A second major shift came with the rise of Scholarly Information Retrieval (IR), as digital libraries and academic search engines scaled. Retrieval moved from manual browsing and citation chasing to systematic indexing and ranking over large corpora, and recommendation systems began to proactively surface relevant papers given a query paper, an author profile, or a research topic. Within Scholarly IR, citation recommendation became a distinct and increasingly formalized problem: instead of "find papers similar to this one," the goal is "given the local writing context, what should I cite here, and why?" This distinction matters because citation recommendation depends not only on topical similarity but also on discourse function and author intent. The emergence of surveys and standardized datasets reflects that citation recommendation has matured into a coherent subfield with shared task definitions and evaluation practices [5].

Progress in literature understanding has also been driven by improvements in data and infrastructure. Large-scale open corpora and machine-readable scholarly metadata have enabled end-to-end pipelines that combine full-text processing, citation links, and structured fields such as authors, venues, affiliations, and concepts. S2ORC provides a large, structured corpus designed to support scientific NLP and document understanding at scale [14], while OpenAlex offers a fully open index and scholarly knowledge graph linking works, authors, venues, institutions, and concepts [17]. Equally important are PDF-to-structure toolchains that extract usable representations from PDFs, including sectioning and bibliographic fields; widely used systems include GROBID [15] and CERMINE [24]. Together, open corpora, open scholarly graphs, and robust parsing tools form the substrate that makes modern deep research operational and auditable.

With these resources in place, Scholarly IR has increasingly moved from purely lexical matching toward neural representation learning. Domain-specific pretraining provides stronger semantic matching than keyword overlap alone [1]. A particularly influential step for literature discovery is paper-level embeddings trained with citation signals. SPECTER is a representative milestone, learning document representations via citation-informed supervision to improve nearest-neighbor retrieval for finding related work at the paper level [2]. Yet better similarity is not the same as diagnostic deep research: similarity-based retrieval can surface plausible candidates, but it does not guarantee that a retrieved paper supports or refutes a specific claim, nor does it localize where evidence is missing. This motivates a second layer: evidence grounding and verification. SciFact frames scientific claim verification as retrieving relevant papers and identifying the exact evidence needed to support or contradict a claim [26]. For auditable systems, this shift from paper-level relatedness to claim-evidence alignment is essential.

In parallel, the community has studied whether systems can compose coherent, properly attributed related-work narratives. Related-work and citation-text generation investigate (i) which prior work to discuss, (ii) what to say about cited papers, and (iii) how to organize these pieces into a structured section. A recent survey synthesizes task formulations, datasets, and evaluation practices in this area [13]. At the same time, moving from retrieval to narrative introduces failure modes that are particularly harmful for scientific writing: hallucinated statements, missing key papers, incorrect attribution, and weak structure [13]. These limitations reinforce the need to treat literature understanding as a structured process with inspectable intermediate artifacts rather than a single long-form generation step.

This paper argues that coverage, novelty, and reproducibility are not solved by generating longer summaries. They require traceable evidence and a multi-stage pipeline that produces auditable intermediate products. We introduce PRIOR ART CHECK, a diagnosis-first system that turns "find related work" into an inspectable sequence of decisions. The pipeline begins with broad candidate

discovery to assemble a large and diverse pool, then performs facet decomposition of the target idea and lightweight abstract-level scoring to prioritize candidates by facet coverage and potential overlap. For shortlisted papers, it escalates to full-text evidence grounding, extracting supporting or conflicting spans that justify each coverage label. The system outputs both a human-readable report and structured artifacts—saved queries, candidate pools, facet-wise coverage labels, extracted evidence snippets, bucketed paper lists, and a synthesis JSON—so the full analysis can be audited and reused.

We also position three complementary modules as natural extensions of the historical trajectory from bibliometrics and scholarly IR to evidence-based, agentic research assistance. First, to support reproducibility assessment, we extract code-like artifacts (code blocks, pseudocode, configuration cues, hyperparameters, and procedural steps) into structured evidence, producing reproducibility signals that reveal what is specified, ambiguous, or missing. Second, to bridge the operational gap between a citation graph and an interpretable story of progress, we modernize science mapping with technology-evolution outputs that highlight milestones, branching lineages, and influential nodes together with a narrative scaffold. Third, because many method differences and empirical claims are best supported by visual assets, we align and organize figures and tables across papers into normalized, queryable assets linked back to sources, enabling evidence-level comparison and more faithful grounding.

We evaluate PRIOR ART CHECK with a paper re-discovery test that probes coverage under limited query information. Concretely, we sample four recent papers (2024–2025) and prompt an LLM to produce a single-sentence description of each paper's core idea without copying the title or abstract; this anonymized sentence is then used as the only query input for three systems: our pipeline, OpenAI Deep Research, and Gemini Deep Research. Each system returns a deduplicated set of candidate papers; we measure whether the original paper is recovered and, for our method, whether it appears in the final top-20 ranking, while also logging whether a failure is due to downstream filtering (the paper was present in the initial seed pool but later removed by lexical/LLM filters). Our runs take about 25 minutes and roughly $0.5 per task, collecting around 210–250 papers during candidate discovery before staged filtering and evidence grounding. Across the four targets, our system recovers all papers in the seed pool (4/4) and ranks three of them in the final top-20 (3/4), whereas OpenAI Deep Research[16] and Gemini Deep Research[7–9] each recover two (2/4). Notably, the returned candidate sets exhibit low pairwise overlap across systems (often only 0–4 shared papers per query, with overlaps such as 2/18, 1/33, and 0/38), indicating that different agents explore substantially different regions of the literature and motivating diagnosis-first, auditable pipelines that preserve intermediate artifacts for inspection and correction.

In addition to paper re-discovery, we report lightweight evaluation results for the two diagnostic tools integrated into PRIOR ART CHECK. For *reproducibility-oriented extraction*, applying the pipeline to the shortlisted papers yields concrete implementation-related artifacts in the majority of cases: across the evaluated set, at least one reproducibility-relevant signal (e.g., code blocks, pseudocode, hyperparameters, dataset descriptions, or procedural steps) is extracted for most papers, with multiple artifact types identified per paper on average. Manual inspection indicates that failures are primarily due to genuinely underspecified experimental descriptions rather than extraction errors, allowing the system to surface reproducibility gaps rather than obscure them. For *technology-evolution mapping*, the generated citation-based evolution graphs consistently identify a small set of milestone papers and coherent branching lineages. Across targets, the resulting graphs exhibit shallow but interpretable depth and limited branching, aligning with known survey-level narratives and highlighting influential nodes that correspond to widely recognized turning points. Qualitative case studies further show that the produced evolution structures support human interpretation by making temporal progression and conceptual divergence explicit. Overall, these results suggest that the additional diagnostic modules produce interpretable and actionable intermediate artifacts, complementing paper-level retrieval by exposing reproducibility signals and historical structure rather than only surface relevance.

In summary, our contributions are: (1) a multi-stage, diagnosis-first framework for prior-art analysis centered on auditability and evidence traceability; (2) reproducibility-oriented extraction that converts scattered implementation cues into structured signals; (3) technology-evolution mapping built on citation networks that yields structured maps and narrative scaffolds; (4) cross-paper figure/table asset alignment to support evidence-level comparison; and (5) a unified suite of structured outputs designed for downstream reuse and analysis.

2 Related Works

2.1 LLM-based Deep Research Agents

Recent efforts have explored autonomous literature review agents that leverage large language models (LLMs) to search, aggregate, and synthesize research at scale. Stanford's STORM system is one representative example: it performs internet-aided research and produces Wikipedia-style articles with structured organization and explicit citations [20]. By decomposing a topic into subquestions and iteratively retrieving sources, STORM demonstrates that LLMs can generate coherent long-form overviews from heterogeneous web evidence. Co-STORM further extends this paradigm by introducing a multi-agent, conversational workflow in which multiple LLM agents collaboratively refine content and incorporate human feedback during the research process [10]. This interaction-oriented design improves coverage breadth and adaptability, especially for open-ended or evolving research questions.

In parallel, open-source systems such as GPT-Researcher exemplify an engineering-driven agentic pattern that plans queries, recursively gathers sources, and synthesizes citation-rich reports with minimal user intervention [4]. Beyond pure literature synthesis, Agent Laboratory connects review to downstream experimentation and report generation, illustrating how agentic systems can participate across multiple stages of the research lifecycle [19, 18]. Collectively, these approaches show that LLM-based agents can dramatically reduce the time required to obtain an initial overview of an unfamiliar domain. However, they primarily operate under a *search-and-summarize* paradigm: success is measured by fluency, topical coverage, and citation inclusion, while deeper questions of evidence sufficiency, systematic omission, and reproducibility are largely left implicit.

2.2 Citation-Based Science Mapping

A complementary line of work arises from bibliometrics and science mapping, which analyze citation networks to uncover the structure and evolution of scientific fields. Early work on citation indexing formalized citations as explicit links between scholarly ideas [6], enabling quantitative analysis of scientific communication. Building on this foundation, bibliographic coupling links papers that share common references [11], while co-citation analysis links papers that are frequently cited together by subsequent work [22]. These primitives have been widely used to cluster research areas, identify influential works, and construct map-like visualizations of scientific domains [21].

Temporal extensions further enrich these analyses by modeling how attention shifts over time. Burst detection techniques identify sudden increases in citation frequency that may correspond to emerging topics, paradigm shifts, or methodological breakthroughs [12]. Such methods are effective for macro-level analysis, revealing trends, lineages, and community structure across large corpora. Nevertheless, citation-based science mapping typically operates at the level of papers and links, without inspecting the internal content of claims. As a result, citation graphs alone cannot guarantee that specific statements are well-supported, nor can they diagnose whether critical technical details or assumptions are absent from the underlying literature.

2.3 Reproducibility and Research Artifacts

Reproducibility has become a central concern in computational research, motivating the development of infrastructures that capture and preserve executable research artifacts. Systems such as ReproZip and ReproServer aim to package code, data, software dependencies, and execution environments so that published experiments can be rerun and verified [23]. Related efforts propose encapsulating experiments as reusable research objects, enabling portability and long-term preservation [25], or designing general frameworks for cross-domain reproducibility of computational studies [3].

These tools substantially improve artifact-level auditability by focusing on execution fidelity and dependency management. However, they are often decoupled from literature understanding workflows. In practice, literature review agents and scholarly question-answering systems may summarize results or compare methods without indicating whether a paper provides sufficient algorithmic detail, hyperparameter specifications, or implementation cues to support reproduction. Consequently, reproducibility assessment is treated as a downstream, manual process rather than an integrated diagnostic signal during literature analysis.

2.4 Evidence-Grounded Claim Verification

Another relevant body of work focuses on evidence-grounded scientific natural language processing, particularly the task of claim verification. SciFact formulates scientific fact-checking as a pipeline that retrieves relevant documents and identifies rationales that support or refute a given claim [26]. By explicitly aligning claims with evidence spans, such systems enhance transparency and auditability, allowing users to inspect why a claim is judged as supported or contradicted.

Despite these advances, claim verification is typically scoped to isolated statements and binary decisions. It does not directly address broader objectives common in prior-art analysis, such as diagnosing facet-wise coverage across a field, reconstructing historical and methodological lineages, or extracting structured signals relevant to reproducibility. As a result, evidence-grounded verification remains a necessary but insufficient component for comprehensive literature auditing.

2.5 Towards a Diagnose-and-Audit Paradigm

Taken together, prior work has made substantial progress in agentic summarization [20, 4, 19], citation-based science mapping [11, 22, 21, 12], reproducibility infrastructure [23, 25], and evidence-grounded claim verification [26]. However, these lines of research remain largely siloed. Summarization agents prioritize narrative fluency, science mapping emphasizes macro-level structure, reproducibility tools focus on executable artifacts, and claim verification targets localized statements.

Gap Analysis and Our Contribution. A key gap emerges at the intersection of these areas: existing systems rarely provide *diagnostic* and *auditable* intermediate representations that allow researchers to systematically assess coverage, trace claims to primary evidence, and evaluate reproducibility signals within a unified pipeline. Our work addresses this gap by adopting a *diagnose-and-audit* paradigm. Rather than producing only a polished summary, we emphasize inspectable intermediate artifacts—such as structured facet decompositions, evidence-aligned mappings, and code- or procedure-level extractions—that enable users to identify missing facets, verify claim support, and reason about reproducibility. This shift positions literature analysis not merely as a generative task, but as a diagnostic process designed for higher-stakes scientific reasoning and downstream decision-making.

3 Methods

3.1 Multi-Stage Prior Art Checking Pipeline

Before retrieval and scoring, the pipeline converts the raw user idea into a compact, structured representation that is reused across all later stages. Concretely, Block 0 produces four artifacts: (i) an *intent record* (user goals and constraints), (ii) a set of *fine-grained technical facets* (F1–Fk), (iii) *structural features* (datasets/metrics/method families), and (iv) *high-level facets* capturing problem/setting/mechanism/assumptions. This shared representation serves as the single source of truth for query generation (Block 1) and overlap estimation (Blocks 2–3).

B0.1 Intent building. We first run `build_prior_art_intent()` to normalize the user request into a machine-readable intent object containing: (1) `user_query` (raw idea text), (2) `user_intent_notes` (what the user is trying to achieve, e.g., novelty checking vs. related-work drafting), and (3) optional `constraints` (e.g., domain, venue preference, recency preference, evaluation emphasis). This intent is propagated into the final synthesis artifacts so that downstream report generation can condition on the user's purpose (e.g., conservative conflict detection vs. broad landscape summary).

B0.2 Fine-grained facets (F1–Fk). Next, `decompose_idea_into_facets()` (prompt ID A.2) extracts $k \in [3, 7]$ fine-grained facets, where each facet is a distinct technical component (task, mechanism, assumption, or contribution). The system assigns stable facet identifiers (F1–Fk) by preserving the returned order and uses these IDs throughout the pipeline (e.g., `covered_facets` outputs in Block 2 and evidence labeling in Block 3). Facets are optimized for *recall*: they are intended to be broad enough that partially overlapping papers can still match at least one facet.

B0.3 Structural features (datasets/metrics/method families). We then extract lightweight "structural cues" with `extract_idea_features()` (prompt ID A.2), yielding three lists: `datasets`, `metrics`, and `method_families`. These features are used in two ways: (i) as query tokens to increase retrieval precision (e.g., adding dataset/metric keywords when present), and (ii) as auxiliary signals for overlap analysis (e.g., `compute_structural_overlap()` compares idea features to paper metadata/abstract content).

B0.4 High-level facets (problem/setting/mechanism/assumption). Finally, `extract_high_level_facets_from_idea()` (prompt ID A.2) produces a structured summary with four categories: `problem_facets`, `setting_facets`, `mechanism_facets`, and `assumption_facets`. This representation supports more interpretable alignment than raw keywords: later, `extract_high_level_facets_from_paper()` (prompt ID A.2) builds the same schema for each candidate paper, enabling `compute_high_level_facet_overlap()` to quantify alignment at the level of task framing, experimental setting, and core mechanisms.

Shared use across Blocks 1–3. The outputs of Block 0 are reused as follows:

- **Block 1 (Retrieval):** facets and features condition query generation (e.g., `generate_search_queries_from_idea()` A.1, `generate_seed_queries_llm()` A.1), ensuring multi-angle coverage of the idea.
- **Block 2 (Abstract scoring):** facet IDs define the schema for `covered_facets`, while high-level facets and structural features provide additional overlap signals beyond lexical matching (e.g., `score_papers_with_abstract_only()` A.2).
- **Block 3 (Evidence):** facets and features seed evidence keyword construction and snippet extraction; refined scoring uses facet definitions to decide roles and conflict likelihood (e.g., `refine_top_papers_with_fulltext()` A.3).

Implementation note (robust parsing). All Block 0 LLM calls are required to return raw JSON. Outputs are parsed with a defensive JSON repair layer (`repair_json_output()`) to handle minor formatting errors, so the downstream pipeline remains deterministic once the structured artifacts are produced.

3.1.1 Block 1: Candidate Discovery (Retrieval and Expansion)

Block 1 constructs an initial candidate corpus ($\sim$200 papers) with high recall. The core design principle is *multi-source, multi-angle retrieval* followed by a *controlled recency balance*: we intentionally mix highly cited foundational work with recent papers to avoid missing emerging directions. The output of Block 1 is a deduplicated set of paper objects in `PipelineState` (`state.papers`, `state.paper_ids`) that will be filtered and scored in Block 2.

B1.1 Seed paper acquisition (optional, high precision). We optionally initialize retrieval with a small set of seed papers from two sources: (i) `seed_papers_from_llm()` (Prompt ID A.1) asks an Expert-tier LLM to recall known relevant papers, producing a JSON list of titles and optional identifiers; and (ii) `seed_papers_from_web_search()` issues web search queries (SerpAPI/DDG) and parses results into paper-like entries using `parse_serpapi_results_with_llm()` and `parse_web_results_with_llm()` (Prompt IDs A.1, A.1). Seed papers are treated as *must-keep* candidates downstream to reduce false negatives from early filtering (see Block 2 prefilter policy).

B1.2 Seed validation and normalization. LLM- or web-derived seeds may contain noisy metadata. We therefore validate and canonicalize seeds via `validate_llm_seeds_via_openalex()`, which resolves each seed title/identifier against OpenAlex and converts it into the internal `Paper` representation (with stable IDs, year/venue/DOI if available). This step also removes nonexistent or unresolvable entries and performs cross-source deduplication (e.g., DOI/arXiv/OpenAlex ID matches, plus normalized title matching).

B1.3 Query generation and exploration loop. To achieve high recall beyond seeds, we generate diversified bibliographic queries from the Block 0 representation. Concretely,

`generate_search_queries_from_idea()` (Prompt ID A.1) produces a small set of keyword-style queries optimized for academic databases (OpenAlex/S2). Optionally, we further diversify query coverage with `generate_seed_queries_llm()` (Prompt ID A.1). These queries are executed in `run_exploration_loop()`, which iteratively retrieves papers, summarizes coverage gaps, and (if enabled) refines queries using `refine_queries_llm()` (Prompt ID A.1). The exploration loop is designed to discover new terminology and recently emerging subtopics not captured by the initial facet list.

B1.4 Dual-bucket retrieval (citation vs. recency). For each query, `retrieve_candidates_with_buckets()` performs a dual-bucket search: a **main bucket** prioritizing overall relevance and citation count, and a **recent bucket** restricting results to papers newer than a configurable threshold (`cfg.recent_year_threshold`, e.g., 2024+). This explicitly prevents the corpus from being dominated by either (i) highly cited older papers or (ii) only the newest papers with sparse citations. Retrieved results are parsed into `Paper` objects and merged into `PipelineState` with deduplication across queries and buckets.

B1.5 Controlled recency quota. After aggregation, `apply_citation_recency_quota()` enforces a target fraction of recent papers in the final candidate set. Specifically, we select a subset such that the proportion of papers with `year` $\geq$ `cfg.recent_year_threshold` lies within [`cfg.recent_min_quota_ratio`, `cfg.recent_max_quota_ratio`] (default 30–60%). The remaining slots are filled with the most cited or otherwise high-signal papers from older years. This policy is crucial for novelty assessment: recent papers reduce the risk of missing concurrent work, while older papers provide stable conceptual baselines.

B1.6 Citation graph expansion (optional, recall booster). Optionally, we expand the candidate corpus using `expand_corpus_via_citation_graph()`, which queries Semantic Scholar for references and citations of high-signal candidates and merges neighbors into the state. This step increases recall for tightly connected literature clusters (e.g., incremental variants or follow-up work) that may not be retrieved by keyword search alone. All newly added nodes pass the same normalization and deduplication logic used for primary retrieval.

Output and handoff to Block 2. The final output of Block 1 is a deduplicated corpus ($\sim$200 papers) stored in `PipelineState`. Each paper contains at minimum `id/title/year/venue/abstract/doi/arxiv_id` when available, enabling the lexical and LLM-based filtering stages in Block 2. The retrieval summary (queries, seed sources, and exploration notes) is also logged and later included in the synthesis artifact for traceability.

3.1.2 Block 2: Abstract-Level Filtering and Scoring

Block 2 is the core scoring stage that reduces the Block 1 corpus ($\sim$200 candidates) to a smaller, high-quality set for full-text refinement. The key design principle is *progressive filtering*: we first apply a fast lexical screen to remove obviously irrelevant papers, then use a lightweight LLM to verify topical relevance, and finally run a structured LLM scoring prompt that estimates overlap with the target idea and attributes overlap to specific facets. The output of Block 2 is a ranked list of `PriorArtMatch` objects containing coverage labels, similarity scores, and interpretable facet-level signals.

Stage 2a: Lexical prefilter (fast, high recall). We first generate a recall-oriented keyword set using `generate_relevance_keywords_from_idea()` (Prompt ID A.2), conditioned on the idea text and the facet list from Block 0. Using these keywords, `simple_relevance_score()` assigns each paper a lightweight score based on title/abstract matches (e.g., higher weight for title matches than abstract matches). We keep papers with `score` > 0 up to a configurable cap (e.g., `cfg.prefilter_max_keep`=80), *and we always retain all seed papers* regardless of lexical score. This "must-keep seeds" rule prevents false negatives caused by vocabulary mismatch or terse abstracts.

Stage 2b: LLM relevance prefilter (boolean gate). Next, `llm_relevance_prefilter()` (Prompt ID A.2) performs a quick, parallelizable binary decision on each retained candidate using only the paper title and abstract. The model returns `{"is_relevant": true/false}`. To maintain recall under occasional LLM formatting noise, we adopt a conservative policy: if the JSON parse

fails or the output is malformed, the paper is *kept* rather than discarded. This stage typically removes $\sim$20–30% of the remaining candidates while preserving ambiguous but potentially important papers.

Stage 2c: Abstract-only scoring (structured overlap estimation). For the final abstract-level scoring pass, we run `score_papers_with_abstract_only()` using a merged prompt (Prompt ID A.2). Given the idea facets (F1–Fk) and the paper metadata (title/venue/year/citations/abstract), the LLM produces a single JSON object containing: (i) `coverage_level` $\in \{0, 1, 2, 3\}$, (ii) `covered_facets` identifying which facet IDs the paper overlaps, (iii) continuous similarity scores `task_similarity` and `method_similarity` in $[0, 1]$, (iv) short explanatory `notes`, and (v) the paper's own high-level facets (`problem/setting/mechanism/assumption`) for interpretable comparisons. If the merged call fails (e.g., invalid JSON), we fall back to a scoring-only prompt (Prompt ID A.2) to still obtain overlap estimates.

Ranking and selection rule. All candidates scored in Stage 2c are converted into `PriorArtMatch` objects and sorted using a lexicographic key that prioritizes overlap and interpretability: **Ranking key (lexicographic):**

1. `coverage_level`
2. `task_similarity`
3. `method_similarity`
4. `facet_overlap`
5. $\min(\texttt{citation_count}, C)$

where `facet_overlap` is computed from the match between `covered_facets` and the idea facet set (optionally combined with high-level facet overlap), and C is a citation cap (e.g., $C = 200$) to prevent extreme citation outliers from dominating ordering among similarly scored papers. This ranking yields an ordered shortlist (typically 60–100 papers) for downstream processing; only the top-K are passed to Block 3 for full-text evidence extraction (via `cfg.fulltext_top_k`).

Outputs. Block 2 produces: (i) a ranked list of matches with structured scores and facet attributions, (ii) per-paper high-level facet summaries for later overlap analysis, and (iii) logging artifacts (kept/filtered counts, parse-failure keeps) to support ablation and debugging.

"'latex

3.1.3 Block 3: Full-Text Refinement and Evidence Extraction

Block 3 upgrades abstract-level matches into *evidence-grounded* assessments by downloading and parsing PDFs for the top-ranked candidates from Block 2, extracting targeted text snippets, and re-evaluating overlap using full-text evidence. The main goal is to reduce false positives/negatives caused by incomplete abstracts and to attach *auditable evidence* to each overlap decision.

B3.1 Top-K selection for full-text analysis. We first select the top-K papers from the Block 2 ranked list according to the ranking key in Section 3.1.2, where $K =$ `cfg.fulltext_top_k` (default 20; reduced in debug mode). This shortlist focuses computational budget on the most likely high-overlap or informative papers.

B3.2 PDF acquisition and section parsing with robust fallbacks. We enrich shortlisted papers with full text using `enrich_papers_with_fulltext()`, which resolves a PDF URL and downloads/parses the document into text chunks and section metadata. To maximize acquisition success, the resolver attempts multiple sources in a fixed priority order (e.g., arXiv $\rightarrow$ Semantic Scholar $\rightarrow$ OpenAlex $\rightarrow$ Unpaywall $\rightarrow$ optional web fallback), and uses retry logic: if a paper fails to download or parse, the system continues with the next-ranked candidates until it collects K papers with usable full text or reaches a maximum attempt budget. Parsed outputs are stored as (i) `paper.chunks` for retrieval and (ii) cached section JSONs (e.g., `pdf_sections/{paper_id}.sections.json`) for later extraction.

B3.3 Evidence keyword construction (facet-conditioned). To guide evidence retrieval, we generate a high-recall keyword set from the idea using `generate_evidence_keywords_from_idea()`, and then expand/normalize it with `make_keywords_for_evidence()` by merging: (i) facet phrases (F1–Fk), (ii) structural features (datasets/metrics/method families), and (iii) optional synonym variants. This produces a single evidence keyword list used consistently across all papers, ensuring comparability of snippet selection.

B3.4 Evidence snippet retrieval via BM25 over chunks. Given `paper.chunks` and the evidence keyword list, `find_evidence_snippets()` performs lexical retrieval (BM25) to identify the top matching passages from each paper. We typically keep the top-m snippets (e.g., $m = 3$) per paper, favoring short paragraphs that are likely to describe the method, experimental protocol, or core assumptions related to the target facets. These snippets are written into run artifacts and later displayed in the report, enabling traceability of overlap judgments.

B3.5 Full-text refinement scoring (coverage + role assignment). Finally, `refine_top_papers_with_fulltext()` re-scores each paper using its abstract *plus* the retrieved evidence snippets (Prompt ID A.3). The model outputs:

- `coverage_level_refined` $\in \{0, 1, 2, 3\}$, which may revise the abstract-only `coverage_level`;
- `role` $\in \{$`main_contribution`, `key_technique`, `background`, `irrelevant`$\}$, describing how the paper should be interpreted relative to the idea; and
- `notes`, a short justification grounded in the provided snippets.

The `role` label is used downstream to prioritize potential conflicts (`main_contribution`) and to separate supporting technique papers (`key_technique`) from broad contextual references (`background`). Papers labeled `irrelevant` are deprioritized even if they appeared superficially similar in abstracts.

Outputs. Block 3 produces: (i) a refined top-K match list with evidence-grounded coverage and role labels, (ii) per-paper evidence snippets (BM25 passages) that support the decision, and (iii) cached full-text artifacts (PDFs and parsed sections). These outputs are consumed by post-processing (key paper selection, intro/RW extraction, synthesis JSON construction) and by the final report renderer. “‘

“‘latex

3.1.4 Block 4: Synthesis and Report Generation

Block 4 converts the scored and evidence-grounded results from Blocks 1–3 into *reusable artifacts* for writing and analysis. Concretely, it (i) extracts facet-aligned Method/Experiment snippets for downstream writing, (ii) selects a small set of representative “key papers” and exports their full Introduction/Related Work text, (iii) consolidates the entire run into a self-contained `synthesis_input.json` with prompt templates, and (iv) renders a human-readable Markdown report with bucketed results and gap analysis.

B4.1 Method/Experiment snippet extraction (Artifact 1.3). We run `build_method_experiment_snippets()` to extract short, facet-relevant paragraphs from the parsed full text of Block 3 papers. For each facet F_i, the system retrieves candidate paragraphs from Method/Experiment sections using BM25 over section text, then applies a lightweight LLM filter (Prompt ID A.4) to (i) decide relevance and (ii) produce a concise bullet summary plus a short supporting snippet. Results are saved as line-delimited JSON: `synthesis/method_exp_snippets.jsonl`, where each record stores `paper_id`, facet label, `section_role` (method/experiment), a bullet, and the cited snippet. This artifact is designed to be directly consumable when writing the Method and Experiments comparisons in a paper draft.

B4.2 Key paper selection and full Intro/RW export (Artifact 1.4). To support high-quality writing grounded in primary sources, we identify a small representative set of papers and export their full Introduction and Related Work sections. Specifically,

`select_key_papers_and_dump_intro_rw_full()` constructs three candidate pools from papers with available full text: (*i*) **top_cited**, (*ii*) **latest**, and (*iii*) **survey/review**. A Heavy-tier LLM (Prompt ID A.4) selects one `primary` and up to two `alternates` per pool to maximize representativeness and viewpoint diversity. For each selected paper, the system extracts complete Introduction and Related Work text using an anchor+window section collector (e.g., capturing "1. Introduction" until "2. Related Work" is detected, including subsections), and stores it as JSON: `synthesis/intro_rw_full/{paper_id}.json`. Selection metadata and per-paper roles are stored in `synthesis/key_papers_selection.json`. This artifact is intended to enable evidence-grounded writing without copying: downstream LLMs use it as supporting context rather than verbatim text.

B4.3 Consolidated synthesis JSON with prompt templates (Artifact 2.1). We then build a single self-contained representation of the run using `build_synthesis_input()`. The resulting `synthesis/synthesis_input.json` includes:

- **metadata**: topic, idea description, facet definitions, task intent, and usage context;
- **global_stats**: corpus sizes, coverage distribution, and facet coverage counts;
- **retrieval_summary**: seeds, queries, sources, and exploration notes for traceability;
- **buckets**: paper IDs bucketed by overlap level (high/partial/adjacent/ignored);
- **papers**: per-paper entries containing overlap scores, facet labels, roles, evidence summaries, and optional full Intro/RW and method/experiment snippets; and
- **prompt_templates**: templates (e.g., full report and short related-work prompts) that can be applied to the same JSON.

This consolidated format makes the pipeline output portable: a downstream LLM can generate a long-form report or a short related-work section by combining `synthesis_input.json` with the corresponding template (e.g., Prompt IDs A.5, A.5).

B4.4 Final Markdown report rendering with gap analysis. Finally, `render_prior_art_report()` produces the human-readable report: `synthesis/prior_art_report.md`. The renderer bucketizes papers primarily by (refined) coverage level (high/partial/adjacent), then sorts within each bucket by similarity and citation signals. Each paper entry includes core metadata (title/venue/year/DOI), covered facets, and selected evidence snippets. In addition, the report includes **facet gap analysis**: for each facet F_i, the system counts how many papers strongly/weakly cover it and highlights under-covered facets as potential novelty opportunities. All rendering functions are pure (no network/LLM calls), ensuring determinism given the stored run artifacts.

Outputs. At the end of Block 4, the run produces four primary deliverables: (i) `method_exp_snippets.jsonl`, (ii) `key_papers_selection.json` and `intro_rw_full/*.json`, (iii) `synthesis_input.json` with prompt templates, and (iv) a final Markdown report `prior_art_report.md`. Together, these artifacts support both systematic novelty checking and efficient, evidence-grounded introduction/related-work drafting. "'

3.1.5 Block 5: System Engineering and Blind Evaluation

Block 5 describes the engineering mechanisms that make the pipeline scalable, stable under API constraints, and suitable for controlled evaluation. We focus on three components: (i) a per-stage concurrency model, (ii) client-layer rate limiting, and (iii) a "blind evaluation" feature that excludes a target paper from the entire pipeline to measure discovery robustness.

B5.1 Concurrency model (per-stage worker pools). The pipeline is implemented as a sequence of stages (Blocks 1–4), each of which may invoke network-bound or LLM-bound operations over many papers. To maximize throughput while keeping control flow simple and debuggable, each stage uses its own `ThreadPoolExecutor` with an explicit worker budget (e.g., `cfg.max_workers_openalex`, `cfg.max_workers_llm_scoring`, `cfg.max_workers_pdf`). Stages are executed sequentially at the block level, but within a stage the work is parallelized over papers/queries. This design has two

practical benefits: (1) concurrency is easy to tune per resource (OpenAlex vs. PDF vs. LLM), and (2) failures are isolated to individual tasks (a single paper download or scoring call), enabling partial progress and robust retries without aborting the entire run.

B5.2 Client-layer rate limiting (sliding-window). External services impose request limits and may throttle or block clients that issue bursts. Rather than adding coarse `sleep` calls at the pipeline level, we enforce rate limits at the *client layer* using a thread-safe sliding-window limiter (`SimpleRateLimiter` in `pipelines/rate_limiter.py`). Each network/LLM task calls:

```
rate_limiter.wait_for_slot()
```

immediately before making its request; the call blocks only if the service-specific budget is exceeded. We maintain separate limiters for each service endpoint, e.g., **OpenAlex** (polite high-throughput), **Semantic Scholar** (more conservative), **LLM** (global safety-net), and **PDF download** (avoid server blocks). This approach preserves parallelism while preventing synchronized bursts from multiple threads. Importantly, rate limiting is always active (including in debug mode), so behavior under debugging matches real runs except for reduced data scale.

B5.3 Failure handling and determinism. Pipeline stages are designed to be resilient to partial failures: (1) transient request errors are naturally smoothed by the limiter and retry logic (notably for PDF resolution in Block 3), (2) individual task failures do not halt the run; instead the system logs failures and continues with the remaining items, and (3) downstream artifacts include fallbacks (e.g., abstracts remain available even if full text is missing), enabling report generation despite incomplete PDF coverage. Rendering functions are pure (no external calls), ensuring deterministic outputs given the stored run state.

B5.4 Blind evaluation via excluded-paper filtering. To evaluate retrieval and ranking quality without leakage, we implement an "excluded paper" feature that removes a target paper (and its identifiers) as if it were unpublished. Users specify excluded entries via `cfg.excluded_papers`, with fields such as title, DOI, arXiv ID, Semantic Scholar ID, or OpenAlex ID. The predicate `is_excluded_paper()` applies a cascade of matching strategies:

- **Exact ID match**: DOI/arXiv/S2/OpenAlex identifiers (case-insensitive);
- **Normalized title match**: Unicode normalization, lowercasing, and alphanumeric filtering;
- **Substring match**: for long titles to tolerate minor subtitle/punctuation variations.

A helper `filter_excluded_papers()` is used to remove excluded items from candidate lists while preserving stable logging and counts.

B5.5 Integration points and traceability. Excluded-paper filtering is applied at multiple points to prevent reintroduction through alternate routes (search results, citation neighbors, or synthesis/report rendering). In particular, filtering is performed during: (i) OpenAlex result parsing, (ii) citation-graph neighbor insertion, (iii) exploration-loop discovery, (iv) seed handling before/after validation, and (v) final artifact construction (synthesis JSON and Markdown report). Each filtering call emits explicit logs of the form `[ExcludedPaper] Filtered X paper(s) from ...`, and the run configuration stores the exclusion list (e.g., `excluded_papers_for_eval`) to ensure experiments are reproducible and auditable.

Evaluation usage. In blind evaluation experiments, we run the pipeline twice: (a) *normal* mode and (b) *excluded* mode. We then compare whether the pipeline can still retrieve and rank close substitutes (papers with similar facets/mechanisms) and whether the facet gap analysis changes materially. This provides a direct test of the system's ability to discover relevant literature independently rather than relying on a known target paper appearing as a seed or a citation-graph neighbor.

3.2 Code Extractor: Implementation-Aware Paper Analysis

Motivation and design goals. Prior-art and literature analysis systems often rely primarily on paper text, yet reproducibility-relevant details are frequently conveyed through *implementation artifacts* that are partially specified or embedded in non-standard formats (e.g., algorithm boxes,

appendix pseudo-code, or code-like fragments scattered across the PDF). Our goal is to provide a *PDF-first* and *format-robust* analysis tool that (i) extracts any available code/pseudo-code evidence, (ii) summarizes implementation-critical signals (frameworks/dependencies/model structure), and (iii) outputs structured completeness and reproducibility diagnostics that are actionable for downstream use (automatic aggregation + human inspection). To this end, we introduce **Code Extractor**, a hybrid rule-based + LLM-assisted module designed with conservative failure handling: it prioritizes deterministic extraction and treats LLM calls as optional enhancements.

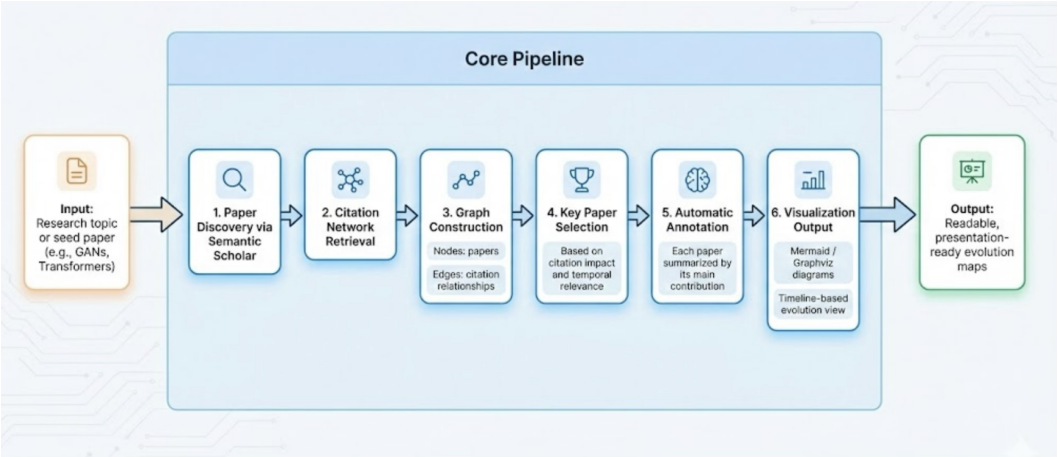


Figure 1: Code Extractor pipeline. Starting from a research paper PDF, the system parses structured text, extracts code and pseudo-code using rule-based methods with optional LLM fallback, performs structured implementation analysis (frameworks, dependencies, and model structure), and outputs both a human-readable Markdown report and a machine-readable JSON summary.

Figure 1 provides an overview of the end-to-end workflow. Concretely, the pipeline is organized into five stages with explicit inputs/outputs and robust fallbacks.

Inputs and outputs. **Input** is a paper PDF (primary), plus optional web-retrieved repository signals when `enable_web_search=True`. The module produces: (i) a **Markdown report** summarizing extracted artifacts and evaluation outcomes, and (ii) a **JSON summary** encoding structured results for downstream tooling. Internally, Code Extractor uses typed result objects to enforce a stable schema, including: `CodeExistenceResult`, `CompletenessResult`, `LLMEvaluationResult`, `FrameworkAnalysisResult`, `TestResult`, and `PaperClassification`.

Stage 1: PDF-to-structured-text parsing. The primary input is a paper PDF. We first convert it into a structured text representation (e.g., Markdown) using an external PDF extraction component. This conversion preserves section cues, formatting, and code-style markers, enabling subsequent extraction to operate on a uniform representation.

Stage 2: Rule-first extraction of code blocks and pseudo-code. Given the parsed text, Code Extractor applies deterministic pattern matching to extract: (i) fenced/formatted code blocks (e.g., ‘‘‘`python ...`‘‘‘ or monospaced listings), (ii) algorithm boxes (e.g., “Algorithm 1” with numbered steps), and (iii) appendix pseudo-code segments via multiple regex templates. This stage outputs `code_blocks` and `pseudocode_blocks`, along with counts and locations. The extraction is deliberately conservative: when uncertain, it prefers to keep candidate blocks rather than discard them.

Stage 3: Selective LLM fallback for recovery and assessment. If rule-based extraction yields insufficient coverage, the module optionally triggers an LLM-assisted pass for (i) recovering pseudo-code structure from narrative descriptions and (ii) evaluating reproducibility-related signals. The LLM is invoked under a conservative predicate (illustrative form):

$$\texttt{should_try_llm} \leftarrow (|C| = 0 \wedge |P| = 0) \vee (|C| > 0 \wedge |P| = 0) \vee (|C| < 2 \wedge |P| = 0) \vee (|\texttt{doc}| > T),$$

where C and P denote extracted code/pseudo-code blocks and T is a long-document threshold. Crucially, LLM calls are *non-blocking for correctness*: if the LLM is unavailable or outputs malformed content, the system falls back to rule-based outputs and continues.

Stage 4: Structured implementation analysis. We normalize extracted artifacts into a structured schema capturing implementation-critical signals:

- **Framework/dependency identification:** When explicit code exists, we parse imports (e.g., via Python AST parsing) to identify frameworks and dependencies; otherwise, we extract framework cues from textual mentions.
- **Model structure summary:** We derive a coarse representation of model components (e.g., layer/module names, parameter hints, or algorithm steps) from code/pseudo-code/text, stored as a structured dictionary.
- **Basic validity checks (when applicable):** For extracted Python snippets, we optionally run syntax validation (e.g., `ast.parse`) and record pass/fail signals in `TestResult`.

Stage 5: Completeness and reproducibility scoring. Code Extractor computes two normalized scores in $[0, 1]$: `completeness_score` and `reproducibility_score`. Intuitively, completeness estimates whether core implementation components are present, and reproducibility estimates whether the provided artifacts support faithful reimplementation. Both scores are accompanied by an explicit `missing_components` list (e.g., training loop, data preprocessing, hyperparameter/config details), producing actionable diagnostics rather than opaque ratings. We also expose a coarse **paper-level classification**:

$$\texttt{category} \in \{\texttt{complete}, \texttt{partial}, \texttt{none}\},$$

Reporting and traceability. The final Markdown report includes (i) existence detection (has code / has pseudo-code, counts), (ii) completeness and missing parts, (iii) LLM-derived evaluation fields when available (framework/dependencies/model structure/missing components), (iv) test signals, and (v) the final classification. The JSON output mirrors the same schema to enable aggregation across papers and integration with other modules.

Robustness and degradation behavior. The module is designed to degrade gracefully under missing signals and heterogeneous formatting:

- If code/pseudo-code cannot be extracted, the system still emits `CodeExistenceResult` and a conservative low-score diagnosis.
- If LLM calls fail, the pipeline falls back to deterministic extraction and static checks.
- Optional web search is treated as *augmentation* and is not required for correctness.

Intended use and integration. Code Extractor is an *analysis tool* that can be applied to any paper corpus selected by an upstream retrieval or filtering stage. It complements text-based similarity signals by providing implementation-aware evidence and reproducibility-oriented diagnostics, supporting both automated processing (JSON) and manual inspection (Markdown).

3.3 Evolution Mapper: Citation-Based Technique Evolution Analysis

Motivation and design goals. Beyond per-paper assessment, users often need an interpretable view of how a research topic evolves: which papers are foundational, how later works branch, and what development paths dominate over time. We introduce **Evolution Mapper**, a citation-based analysis tool that builds a directed citation network from a topic query or seed paper and produces (i) a compact evolution map suitable for inclusion in reports (Mermaid graph) and (ii) a detailed Markdown analysis (key papers, timelines, and evolution paths). The module emphasizes *transparency* and *auditability*: edges are grounded in citations rather than implicit similarity, and all outputs remain explainable via bibliographic relations.

Figure 2 outlines the main stages. The Evolution Mapper consists of four subroutines: paper discovery, citation-graph construction/expansion, key-paper selection and timeline organization, and evolution-path extraction with visualization.

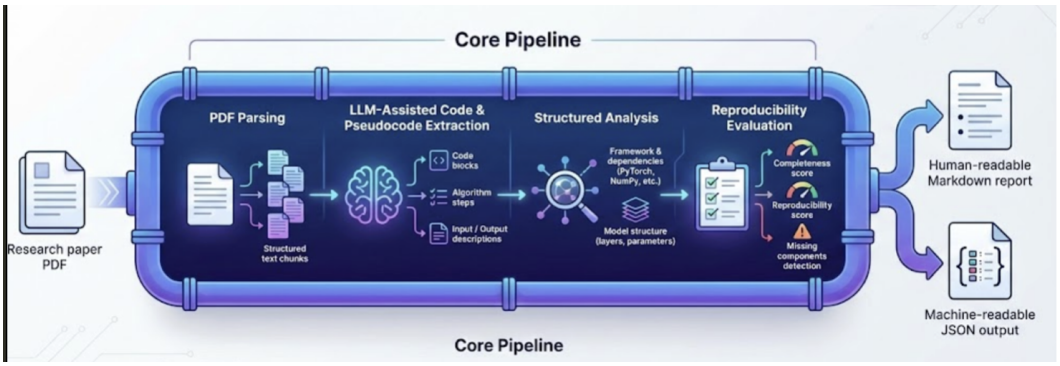


Figure 2: Evolution Mapper pipeline. Given a research topic or seed paper, the system discovers papers, retrieves citation metadata, constructs a directed citation graph, selects key papers based on impact and temporal relevance, annotates nodes with concise contributions, and outputs a readable evolution map (e.g., Mermaid/Graphviz diagrams and timeline views) along with a detailed Markdown report.

Inputs, internal representation, and outputs. **Input** is either (i) a topic query or (ii) a seed paper identifier, optionally with constraints such as maximum paper budget and year range. Internally, the module maintains a citation network: nodes are papers with metadata (`title, year, venue, authors, abstract, citation_count`), and directed edges encode citation relations. **Outputs** include: (i) a Mermaid graph for visualization, (ii) a Markdown report with statistics and paths, and (iii) an internal structured representation (`papers` and `edges`) for downstream use.

Stage 1: Paper discovery. The mapper first retrieves an initial set of relevant papers using a bibliographic search service. This stage returns a bounded candidate list (e.g., top-N by relevance/-coverage), together with stable identifiers for subsequent citation retrieval.

Stage 2: Citation metadata retrieval and graph construction. For each discovered paper, we retrieve two lists: **references** (papers cited by the current paper) and **citations** (papers citing the current paper). We then construct a directed edge list using the semantics:

$$(u \to v) \text{ means } u \text{ cites } v.$$

The graph is expanded by inserting newly observed nodes that appear in references/citations, enabling discovery beyond the initial query result set. This yields a connected citation neighborhood that can be substantially larger than the seed set.

Stage 3: Temporal constraints and timeline organization. To interpret the citation network as an evolution view, we apply temporal constraints during analysis: when extracting evolution paths, we only traverse to nodes with strictly increasing publication years. We also organize papers into a timeline view by year, which enables (i) inspection of research bursts and (ii) alignment of major developments with time.

Stage 4: Key paper selection. To keep outputs readable, we select a subset of key papers using a simple, transparent policy that balances *impact* and *temporal coverage*. Concretely, we rank candidates by citation signals (e.g., citation count) with safeguards to avoid dominance by a single era, and we ensure key papers are distributed across the covered year range. The report includes a ranked list of key papers with metadata and short summaries.

Stage 5: Evolution path extraction (DFS) and branching analysis. Given the directed graph and temporal constraint, we extract representative evolution paths using depth-first traversal starting from early influential works or the chosen seed paper. We record branching points (nodes with multiple temporally valid outgoing edges) as indicators of diversification. These paths form the basis for a narrative "how the field evolved" summary in the Markdown report.

Stage 6: Visualization and reporting. We render a visualization-ready evolution map in Mermaid format, where nodes are labeled with paper titles (optionally shortened) and edges follow citation direction. The accompanying Markdown report provides: network statistics (node/edge counts, year span), a key-paper table, year-by-year timeline grouping, and extracted evolution paths with brief annotations.

Intended use and integration. Evolution Mapper is an *analysis tool* that can be run independently of upstream retrieval strategies. When combined with a prior-art discovery pipeline, it provides a complementary macro-level perspective: rather than only ranking papers by overlap, it explains how relevant work is connected and how ideas propagate and branch over time, helping users structure related-work narratives and identify influential lineages.

4 Experiment

4.1 Prior-Art Retrieval Evaluation via Paper Re-identification

Metric	Score
Ours @Top20	3/4
Ours (SeedRecall)	4/4
OpenAI DR	2/4
Gemini DR	2/4

Query 1

	OpenAI(8)	Gemini(18)
Ours(30)	2/8	2/18
OpenAI(8)	–	3/18
Gemini(18)	3/8	–

Query 2

	OpenAI(18)	Gemini(33)
Ours(56)	3/18	2/33
OpenAI(18)	–	1/33
Gemini(33)	1/18	–

Query 3

	OpenAI(14)	Gemini(24)
Ours(21)	1/14	2/24
OpenAI(14)	–	4/24
Gemini(24)	4/14	–

Query 4

	OpenAI(7)	Gemini(38)
Ours(56)	0/7	0/38
OpenAI(7)	–	0/38
Gemini(38)	0/7	–

Table 1: **Paper re-identification evaluation.** Left: recovery of the target paper over 4 queries. Right: pairwise overlap between agents' retrieved sets for each query. Each entry is reported as $|R_i \cap R_j|/|R_j|$ (row agent i, column agent j); numbers in parentheses denote each agent's returned list size. **SeedRecall** ("Ours seed") means the target paper appears in our seed list but may still be removed later by lexical/LLM prefiltering.

Task. We evaluate whether a deep-research system can *re-identify* a target paper given only a short natural-language description of its core contribution. We select four target papers published in 2024–2025. For each target, we use an LLM to produce a *single-sentence* description of the core idea under a strict non-copying constraint (no verbatim spans from the title or abstract), and use this sentence as the sole query for retrieval.

Compared systems and outputs. We compare **Ours** (the proposed multi-stage prior-art pipeline) against **OpenAI Deep Research** and **Gemini Deep Research**. Each system returns a ranked list of candidate papers; we record the returned list size N for each query (shown in parentheses in Table 1). For our system, a full run collects roughly 210–250 initial candidates in Block 1 and then progressively filters and re-ranks them; empirically, one run takes about 25 minutes and costs approximately $0.5.

Target matching and recovery metrics. A retrieved entry is matched to the target paper by normalized identifiers when available (DOI/arXiv/OpenAlex/S2); otherwise we fall back to normalized title matching (lowercased, punctuation removed). For **Ours**, we report **Recover@20**, i.e., whether the target paper appears in the final top-20 list displayed in the report. For the baseline systems, we report recovery if the target appears anywhere in their returned list for that query.

Seed recall vs. seed survival ("Ours seed"). To diagnose where failures occur in our pipeline, we additionally report **SeedRecall** (denoted as **Ours seed** in Table 1): whether the target paper is present in the output of `seed_papers_from_llm()` *before* Block 2 filtering. A target paper can be recalled as a seed yet later removed by Stage 2a lexical prefiltering or Stage 2b LLM relevance filtering. We

refer to this case as a **seed-survival failure**, indicating that early discovery succeeded but the filtering stage produced a false negative.

Cross-system overlap. To characterize behavioral diversity across systems, we compute a *column-normalized overlap* between retrieved sets. For result sets R_i (row) and R_j (column), each matrix entry is reported as $|R_i \cap R_j|/|R_j|$, i.e., the fraction of the column system's results that also appear in the row system's list.

Results. Table 1 shows that our system recovers 3/4 targets in the final top-20 list, while OpenAI Deep Research and Gemini Deep Research each recover 2/4. Notably, **SeedRecall** is 4/4 for our system, suggesting that the remaining miss is due to false negatives introduced by lexical/LLM filtering rather than insufficient candidate discovery. Across the four queries, pairwise overlaps are generally low (and can be near-zero in one case), indicating that different deep-research systems may retrieve largely disjoint candidate sets even under the same query formulation.

4.2 Code Extractor Evaluation

Task Definition. The Code Extractor aims to automatically recover structured, implementation-level information from machine learning research papers. Rather than reproducing executable code, the tool evaluates whether a paper provides sufficient algorithmic and implementation details to support faithful re-implementation. Given a paper, the extractor outputs a structured record describing pseudocode completeness, reproducibility level, implementation frameworks, software dependencies, model structure, and missing implementation components.

Ground Truth Construction. To enable systematic evaluation, we construct an manually annotated ground truth dataset under the assistance of LLM, consisting of 10 representative machine learning papers. The selected papers span a diverse range of research areas, including neural architectures, AutoML systems, evolutionary methods, and machine learning systems. For each paper, we manually annotate a structured record capturing: (i) pseudocode completeness, (ii) reproducibility level, (iii) implementation frameworks, (iv) software dependencies, (v) model structure descriptions, and (vi) missing implementation components. Annotations are produced through careful inspection of the paper body, appendices, and referenced repositories when available, following a consistent rubric to ensure cross-paper comparability.

Evaluation Protocol. The evaluation compares the structured outputs generated by the Code Extractor against the manually constructed ground truth. Performance is assessed along two complementary dimensions. First, for ordinal scores such as pseudocode completeness and reproducibility, we evaluate whether the extractor assigns values consistent with expert annotations, focusing on relative correctness rather than exact numerical equality. Second, for categorical fields such as frameworks, dependencies, and missing components, we evaluate set-level agreement between extracted outputs and ground truth annotations. Model structure descriptions are evaluated qualitatively, examining whether the extractor correctly identifies the architectural family and its key components.

Case Study. We illustrate the extractor's behavior using *Attention Is All You Need* as a representative example. The extractor correctly identifies the Transformer as a multi-layer encoder–decoder architecture and detects TensorFlow as the primary implementation framework, along with key dependencies such as `tensor2tensor`. Consistent with expert annotations, the extractor flags the absence of a complete training loop and dataset preprocessing pipeline, reflecting partial but non-complete reproducibility.

Aggregate Results. Table 2 summarizes the evaluation results of the Code Extractor across all ground truth papers.

Results Analysis. The Code Extractor demonstrates strong agreement with expert annotations on both ordinal scores and categorical fields. Framework and dependency extraction achieve near-perfect alignment, while pseudocode completeness and reproducibility scores closely track human judgments. Importantly, the extractor consistently identifies missing implementation components that limit reproducibility, without hallucinating non-existent code artifacts. Most discrepancies

Paper	Pseudo.	Reprod.	Framework	Deps.	Missing Components (GT)
AutoML-Zero	1	2	TensorFlow	numpy, scipy, tf	Training hyperparameters; preprocessing; search space
Attention Is All You Need	2	3	TensorFlow	numpy, tf, tensor2tensor	Training loop; dataset preprocessing
Relative Position Self-Attention	2	2	TensorFlow	numpy, tf	End-to-end training; preprocessing; tuning
Privileged Zero-Shot AutoML	1	1	PyTorch	numpy, torch, sklearn	Hyperparameters; architecture; training loop
AI Conference Code Availability Study	0	2	–	–	No algorithmic pseudocode; no reproduction protocol

Table 2: End-to-end evaluation of the Code Extractor on all annotated ground truth papers, reporting ground truth scores and annotations. Pseudo.: pseudocode completeness score. Reprod.: reproducibility score.

Paper	Pseudo.	Reprod.	Framework	Deps.	Missing Components (GT)
SystemDS	1	2	Java / C++ / Python	java, numpy, scipy	End-to-end examples; hyperparameter search
Evolving Unsupervised DNNs	2	1	TensorFlow	numpy, tf	Architecture config; optimizer; preprocessing
DarwinML	2	2	Python (sklearn)	numpy, scipy, sklearn	Pipeline execution; experimental setup
ML + Human Knowledge	1	1	PyTorch	numpy, torch	Architecture; training; evaluation
Neural Programs	2	2	PyTorch	numpy, torch	Data pipeline; hyperparameters; evaluation code

Table 3: Table 2 (continued).

arise in borderline cases where papers provide partial algorithmic descriptions without explicit implementation boundaries, in which the extractor adopts a conservative diagnostic stance.

4.3 Evaluation of Envolution Map

4.3.1 Evaluation Objective

The Envolution Map is designed as a conceptual analysis tool that reconstructs and visualizes the evolutionary trajectory of research ideas from scientific literature. Unlike predictive models, its output is a structured evolution network rather than numerical predictions. Consequently, the evaluation focuses on structural alignment, coverage of influential works, and the additional analytical value provided by the tool.

Specifically, we aim to answer the following questions: (i) whether Envolution Map can recover the core evolutionary structure of a research topic, (ii) how closely the generated network aligns with an established external reference, and (iii) whether it provides additional insights beyond existing citation-based tools.

4.3.2 Ground Truth Construction

To obtain an independent and widely accepted reference, we adopt *Connected Papers* as the ground truth for evaluation. Connected Papers constructs literature graphs based on citation relationships and semantic similarity, and is commonly used for exploratory literature review.

For each topic, we select a canonical seed paper and generate a Connected Papers graph containing 41 representative publications. The graph is treated as the ground truth evolutionary structure.

Figure 3 illustrates an example ground truth evolution relationship diagram for the ImageNet-based CNN architecture evolution topic.

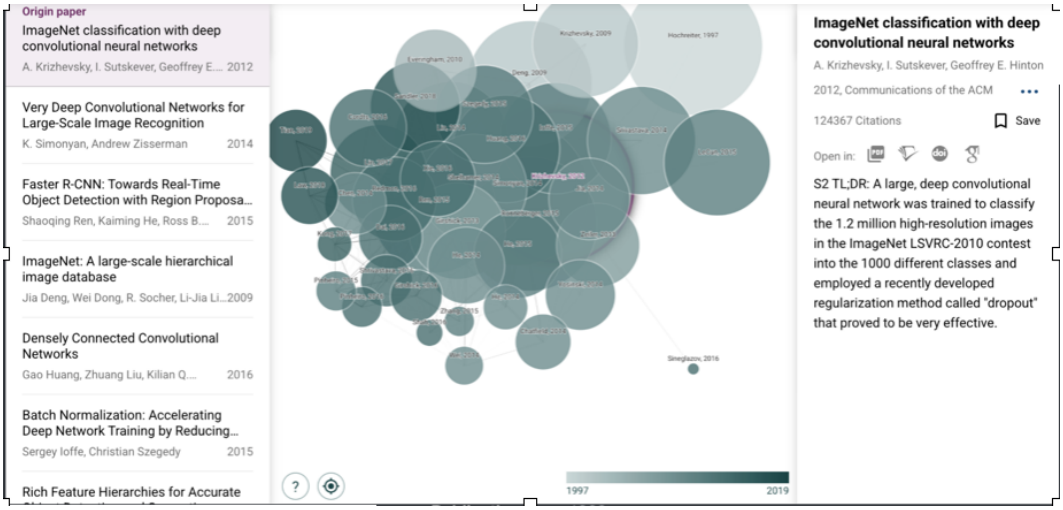


Figure 3: Ground truth evolution relationship diagram generated by Connected Papers for ImageNet and CNN architecture evolution. Nodes denote key papers, and edges represent citation-based evolutionary relationships.

4.3.3 Envolution Map Output

Using the same seed paper and topic, we apply the Envolution Map tool to generate a research evolution network. Unlike Connected Papers, Envolution Map is not restricted to a fixed graph size and emphasizes broader paper coverage, richer metadata, and interactive visualization.

Figure 4 shows the Envolution Map output for the same ImageNet topic.

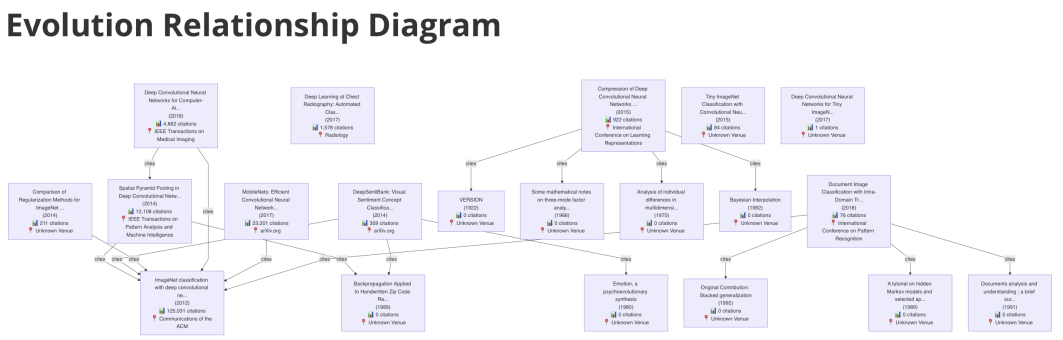


Figure 4: Evolution map generated by the Envolution Map tool for the ImageNet architecture evolution topic.

4.3.4 Evaluation Dimensions and Protocol

We evaluate Envolution Map along five dimensions, each scored on a 0–10 scale by an LLM-based judge (ChatGPT), comparing the generated output against the Connected Papers ground truth:

- **Paper Coverage**: coverage of key papers appearing in the ground truth.
- **Citation Accuracy**: correctness of citation and evolutionary relationships.
- **Network Similarity**: structural similarity between the generated network and the ground truth graph.
- **Additional Value**: usefulness beyond Connected Papers, such as richer visualization, broader coverage, or improved interpretability.
- **Overall Score**: an aggregated assessment of the overall quality.

Topic	Seed Paper	Ground Truth	Overall	Coverage	Citation	Network	Add. Value	Std.
Transformers	Attention Is All You Need (2017)	41 (1997–2024)	6.5	5.5	6.5	5.5	**8.5**	0.31
GAN	Generative Adversarial Nets (2014)	41 (2012–2023)	6.8	6.0	6.5	6.0	**8.5**	0.0
ImageNet	ImageNet Classification with Deep CNNs (2012)	41 (1997–2019)	**7.5**	7.0	**8.0**	7.0	**9.0**	0.0
Average	–	–	**6.9**	6.2	**7.0**	6.2	**8.7**	–

Table 4: Table 4: Evaluation results of Envolution Map across three research themes. For each theme, the ground truth consists of 41 representative papers selected by Connected Papers, spanning the indicated publication years.

Each topic is evaluated multiple times, and we report the mean score and standard deviation.

4.3.5 Overall Quantitative Results and Analysis

Table 4 summarizes the evaluation results across three research themes.

Transformers in NLP. Using *Attention Is All You Need* (2017) as the seed paper, the ground truth contains 41 papers spanning 1997–2024. Envolution Map achieves an overall score of 6.5/10. While it provides a substantially broader overview (over 1000 papers), several key works identified by Connected Papers (e.g., RoBERTa and T5) are missing, and citation relationships are not fully aligned. Nevertheless, its visualization and expanded coverage yield high additional value (8.5/10).

Generative Adversarial Networks. For the GAN topic, Envolution Map covers recent developments and provides richer metadata compared to Connected Papers. However, coverage of seminal works and citation consistency remain limited. The tool achieves an overall score of 6.8/10, with a high additional value score of 8.5/10.

ImageNet and CNN Architecture Evolution. The ImageNet topic yields the strongest performance, with an overall score of 7.5/10. Envolution Map achieves the highest citation accuracy (8.0/10) and additional value (9.0/10), offering a comprehensive and interpretable view of CNN architecture evolution.

Overall, Envolution Map demonstrates strong **additional analytical value** (average 8.7/10), consistently outperforming the reference system in terms of visualization and breadth of coverage. Citation accuracy is generally good (average 7.0/10), while network similarity and paper coverage show room for further improvement. Among the evaluated topics, ImageNet-based CNN evolution exhibits the most robust performance across all dimensions.

References

[1] Iz Beltagy, Kyle Lo, and Arman Cohan. SciBERT: A pretrained language model for scientific text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019. URL `https://aclanthology.org/D19-1371/`.

[2] Arman Cohan, Iz Beltagy, Kyle Lo, and Daniel Weld. SPECTER: Document-level representation learning using citation-informed transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL `https://aclanthology.org/2020.acl-main.207/`.

[3] Lázaro Costa, Susana Barbosa, and Jácome Cunha. A framework for supporting the reproducibility of computational experiments in multiple scientific domains. *arXiv preprint arXiv:2503.07080*, 2025.

[4] Assaf Elovic et al. GPT-Researcher: An open-source deep research agent, 2025. URL `https://github.com/assafelovic/gpt-researcher`. GitHub repository (accessed 2025-12-13).

[5] Michael Färber and Adam Jatowt. Citation recommendation: Approaches and datasets. *International Journal on Digital Libraries*, 2020. doi: 10.1007/s00799-020-00288-2. URL `https://link.springer.com/article/10.1007/s00799-020-00288-2`.

[6] Eugene Garfield. Citation indexes for science: A new dimension in documentation through association of ideas. *Science*, 122(3159):108–111, 1955. doi: 10.1126/science.122.3159.108.

[7] Google. Gemini deep research agent. `https://ai.google.dev/gemini-api/docs/agents/deep-research`, 2025. Accessed: 2025-12-13.

[8] Google. Deep research is now available on mobile. `https://blog.google/products/gemini/google-gemini-deep-research-mobile/`, 2025. Accessed: 2025-12-13.

[9] Google. Use deep research in gemini. `https://support.google.com/a/users/answer/16098300`, 2025. Accessed: 2025-12-13.

[10] Yucheng Jiang, Yijia Shao, Dekun Ma, Sina J. Semnani, and Monica S. Lam. Into the unknown unknowns: Engaged human learning through participation in language model agent conversations, 2024. URL `https://arxiv.org/abs/2408.15232`. EMNLP 2024 Main; arXiv:2408.15232.

[11] M. M. Kessler. Bibliographic coupling between scientific papers. *American Documentation*, 14(1):10–25, 1963. doi: 10.1002/asi.5090140103.

[12] Jon Kleinberg. Bursty and hierarchical structure in streams. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '02)*, pages 91–101, 2002. doi: 10.1145/775047.775061.

[13] Junyi Jessy Li, Jessica Ouyang, David Bamman, Kyle Lo, Arman Cohan, and Yejin Choi. Related work and citation text generation: A survey. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 13723–13774, Miami, Florida, USA, November 2024. Association for Computational Linguistics. URL `https://aclanthology.org/2024.emnlp-main.767/`.

[14] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, et al. S2orc: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL `https://aclanthology.org/2020.acl-main.447/`.

[15] Patrice Lopez. Grobid: Combining automatic bibliographic data recognition and term extraction. In *Proceedings of the 13th European Conference on Research and Advanced Technology for Digital Libraries (ECDL)*, 2009. doi: 10.1007/978-3-642-04346-8_62.

[16] OpenAI. Deep research system card. `https://openai.com/index/deep-research-system-card/`, 2025. Accessed: 2025-12-13.

[17] Jason Priem, Heather Piwowar, and Richard Orr. Openalex: A fully-open index of scholarly works, authors, venues, institutions, and concepts, 2022. URL `https://arxiv.org/abs/2205.01833`.

[18] Samuel Schmidgall and Contributors. AgentLaboratory: End-to-end autonomous research workflows. `https://github.com/SamuelSchmidgall/AgentLaboratory`, 2025. Accessed: 2025-12-13.

[19] Samuel Schmidgall et al. Agent laboratory: Using llm agents as research assistants, 2025. arXiv:2501.04227.

[20] Chenghao Shao et al. Storm: Assisting in writing wikipedia-like articles from scratch with large language models, 2024. arXiv:2402.14207.

[21] Richard M. Shiffrin and Katy Börner. Mapping knowledge domains. *Proceedings of the National Academy of Sciences*, 101(suppl_1):5183–5185, 2004. doi: 10.1073/pnas.0307852100.

[22] Henry Small. Co-citation in the scientific literature: A new measure of the relationship between two documents. *Journal of the American Society for Information Science*, 24(4):265–269, 1973. doi: 10.1002/asi.4630240406.

[23] Vicky Steeves, Ricardo Rampin, and Fernando Chirigati. Reproducibility, preservation, and access to research with reprozip and reproserver. *IASSIST Quarterly*, 44(1–2):1–11, 2020.

[24] Dominika Tkaczyk, Paweł Szostek, Mateusz Fedoryszak, Łukasz Dendek, and Krzysztof Bolikowski. Cermine: Automatic extraction of metadata and references from scientific literature. *International Journal on Document Analysis and Recognition (IJDAR)*, 18(4):317–335, 2015. doi: 10.1007/s10032-015-0249-8.

[25] Dinh-Hoan Ton That, Guillaume Fils, Zesheng Yuan, and Tanu Malik. Sciunits: Reusable research objects. In *2017 IEEE 13th International Conference on e-Science (e-Science)*, pages 374–383. IEEE, 2017.

[26] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, and Arman Cohan. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL `https://aclanthology.org/2020.emnlp-main.609/`.

A Prompt Library for the Prior Art Check Pipeline

This appendix documents the prompt templates used by the `Prior Art Check` multi-stage pipeline. Each prompt is assigned a stable Prompt ID for traceable reference in the system description and ablation studies. Placeholders (e.g., `{idea_description}`) are filled at runtime.

Notation. `LLM role` indicates the model tier used (Light/Expert/Heavy). All prompts below require **machine-readable JSON outputs** (unless stated otherwise), enabling deterministic parsing and downstream aggregation.

A.1 Block 1: Candidate Discovery Prompts

B1-P1: Seed Paper Recall

Function: `seed_papers_from_llm()` **LLM role:** Expert **Output:** JSON array of paper metadata (title/venue/year/doi/arxiv/url)

```
You are an expert researcher.
Your task is to recall real academic papers that are closely related
to the following research idea:
"""{idea_description}"""
Requirements:
- Only include real papers that you reasonably believe exist.
- Prefer papers that describe methods, models, or techniques that
  would be informative when evaluating the novelty of this idea.
- If you are uncertain about a detail (e.g., exact year or venue),
  you may set that field to null.
For each paper, provide:
- title: the paper's title (string)
- venue: conference or journal name, or null if unknown
- year: integer publication year, or null if unknown
- doi: string or null
- arxiv_id: string (e.g., "2104.12345") or null
- url: string or null (e.g., publisher or arXiv URL)
Return ONLY raw JSON, no backticks, in the following format:
[
  {
    "title": "...",
    "venue": "...",
    "year": 2024,
    "doi": "10.xxx/yyy" or null,
    "arxiv_id": "2312.00752" or null,
    "url": "https://..." or null
  },
  ...
]
Constraints:
- At most {max_seeds} papers.
- Do NOT include any commentary outside the JSON.
JSON:
```

B1-P2: Web Search Query Generation

Function: `generate_web_search_queries()` **LLM role:** Expert **Output:** `{"queries": [...]}`

```
You are an expert researcher.
Your task is to design web search queries that will help find
academic papers related to the following research idea:
"""{idea_description}"""
Requirements:
- Each query MUST be suitable for Google or DuckDuckGo web search.
- Each query should look like what a researcher would actually type,
```

```
  combining key technical phrases from the idea.
- Focus on techniques, methods, or key concepts mentioned in the idea.
- Do NOT restrict the domain explicitly (e.g., do not add generic field names
  unless they clearly appear in the idea itself).
- Do NOT add site filters such as "site:arxiv.org" or date filters.
  These will be handled later by the system.
- IMPORTANT: Do NOT use quotation marks ("") in queries. Write plain keywords
  without exact-match operators, as they severely limit search results.
- Avoid overly generic queries (e.g., "deep learning method").
- Aim for diverse queries that cover different angles of the idea.
- At most {max_queries} queries.
Respond with ONLY valid JSON, no backticks:
{"queries": ["query1", "query2", ...]}
```

B1-P3: Parse SerpAPI Results into Paper Candidates

Function: `parse_serpapi_results_with_llm()` **LLM role:** Light **Output:** JSON array of extracted paper entries

```
You are a helpful research assistant.
You are given:
- A research idea: {idea_description}
- A web search query: {query}
You also have search results (from multiple modes), in JSON:
{results_str}

Your task:
From these results, identify up to {max_papers} entries that are MOST LIKELY to be
    academic papers relevant to the idea.
Prefer results that clearly look like academic papers (e.g., conference proceedings,
     journal articles, arXiv papers, open-access repositories).
For each selected result, output:
title: string (the paper title, cleaned up)
year: integer if you can guess from the snippet/title/URL, else null
venue: short string (e.g., conference name, journal name, "arXiv"), or null if
    unknown
url: the link
source_mode: which search mode it came from (one of: all_non_arxiv, all_arxiv,
    range_non_arxiv, range_arxiv)
reason: 1-2 sentences why this seems relevant to the idea.
Output format: Return ONLY a JSON array of objects with the above fields. No extra
    commentary.

JSON:
```

B1-P4: Parse DuckDuckGo Results into Paper Candidates

Function: `parse_web_results_with_llm()` **LLM role:** Light **Output:** JSON array of extracted paper entries

```
You are a helpful research assistant.

You are given:
A research idea: {idea_description}
You also have web search results in JSON:
{results_str}

Your task:
From these results, identify up to {max_papers} entries that are MOST LIKELY to be
    academic papers relevant to the idea.
Prefer results that clearly look like academic papers (conference proceedings,
    journal articles, arXiv papers).
For each selected result, output:
```

```
title: string (the paper title, cleaned up)
year: integer if you can guess from the snippet/title/URL, else null
venue: short string (e.g., conference name, journal name, "arXiv"), or null if
    unknown
url: the link
source_mode: the source_mode from the input (e.g., serpapi_all_years, ddg_arxiv)
reason: 1-2 sentences why this seems relevant to the idea.
Output format: Return ONLY a JSON array of objects with the above fields. No extra
    commentary.

JSON:
```

B1-P5: Bibliographic Query Generation for OpenAlex/S2

Function: `generate_search_queries_from_idea()` **LLM role:** Expert **Output:** JSON list of strings `["q1", "q2", ...]`

```
You are an expert researcher.

Your task is to design bibliographic search queries for academic paper databases (e.
    g., OpenAlex, Semantic Scholar) that are relevant to the following research
    idea:

"""{idea_description}"""

Requirements:
Each query should be a short keyword-style query (not a full sentence).
Combine key technical terms from the idea, including:
problem/task,
key methods or mechanisms,
important properties or techniques.
Do NOT add year or venue constraints in the query. (The system will handle year
    filters separately.)
Do NOT add site filters (e.g., "site:arxiv.org").
Queries should be reasonably distinct to cover different angles.

CRITICAL RULES for query formatting:
Each query should use at MOST ONE exact phrase in quotes
Remaining keywords should be unquoted, separated by spaces
Avoid multiple quoted phrases in the same query (this kills recall)
Mix broad queries (core concept) with more specific queries (concept + 1-2 keywords)
Keep queries simple and focused
Return ONLY raw JSON list of strings: ["query 1", "query 2", ...]

Constraints:
At most {num_queries} queries.
Do NOT include any commentary outside the JSON.
JSON:
```

B1-P6: Diversified Seed Query Generation

Function: `generate_seed_queries_llm()` **LLM role:** Expert **Output:** JSON list of strings

```
You are generating search queries for academic paper retrieval.

Research topic: {topic}
Research idea: {idea[:600]}
Key facets: {facet_text}

TASK: Generate {max_queries} short, diversified search queries suitable for Google
    Scholar / arXiv / Semantic Scholar.

Guidelines:
```

```
Keep each query SHORT (3-8 words, like search engine queries)
Cover different angles: task/problem, method, technique, application
Include at least 1-2 queries with recent keywords or year hints (2023, 2024)
Avoid long natural language sentences
Do NOT use quotes or boolean operators
Output strictly valid JSON list of strings: ["query1", "query2", ...]
```

B1-P7: Hierarchical Query Refinement

Function: `refine_queries_llm()` **LLM role:** Heavy **Output:** JSON list of strings

```
You are refining search queries to find MORE relevant papers, especially RECENT ones
     (2023-2024).

Research topic: {topic}
Idea: {idea[:400]}
Key facets: {facet_text}
Papers already found: {paper_context}

TASK: Generate {cfg.max_refine_queries} additional SHORT search queries that:
Cover angles NOT well-represented in the papers above
Use synonyms or alternative terminology
Include recent buzzwords (foundation models, diffusion, etc. if applicable)
Target recent papers (add year hints like "2024" if helpful)
Output strictly valid JSON list of strings: ["query1", "query2", ...]
```

A.2 Block 2: Abstract-Level Scoring Prompts

B2-P1: Idea-to-Facets Decomposition

Function: `decompose_idea_into_facets()` **LLM role:** Light **Output:** JSON list of facet strings

```
You are a JSON generator. Do NOT call any tools or write/run any code. Think
    silently and then output the final answer as JSON only. Return raw JSON only,
    without backticks or any surrounding text.

Given the following research idea, please extract 3-7 key technical facets that
    characterize this idea.

A "facet" is a distinct technical component, method, assumption, or contribution of
    the idea.

Research Idea: {idea_description}

Please output ONLY a JSON list of strings, where each string is one facet. Example
    format: ["facet 1", "facet 2", "facet 3"]

JSON output:
```

B2-P2: Idea Feature Extraction (Datasets/Metrics/Method Families)

Function: `extract_idea_features()` **LLM role:** Light **Output:** JSON object with `datasets/metrics/method_families`

```
You are a JSON generator. Do NOT call any tools or write/run any code. Think
    silently and then output the final answer as JSON only. Return raw JSON only,
    without backticks or any surrounding text.

Given the following research idea, please extract the key structural features:

Datasets (e.g., KITTI, nuScenes, ImageNet)
```

```
Metrics (e.g., mAP, F1, Accuracy)
Method Families (e.g., Transformer, CNN, Diffusion)
Research Idea: {idea_description}

Please output ONLY a JSON object with keys "datasets", "metrics", "method_families".
     Each value should be a list of strings. Example: {"datasets": ["KITTI"], "
    metrics": ["mAP"], "method_families": ["Transformer"]}

JSON output:
```

B2-P3: High-Level Facets from Idea

Function: `extract_high_level_facets_from_idea()` **LLM role:** Light **Output:** JSON object with problem/setting/mechanism/assumption facets

```
You are a JSON generator. Do NOT call any tools or write/run any code. Think
    silently and then output the final answer as JSON only. Return raw JSON only,
    without backticks or any surrounding text. Do NOT include any comments,
    trailing commas, or extra tokens. Only output valid JSON.

Given the following research idea, please extract high-level structural facets.

Facet Types:
problem_facets: What specific problem/task is it solving? (e.g., "monocular 3D
    detection", "long-horizon sequence modeling")
setting_facets: What is the setting, modality, or protocol? (e.g., "indoor RGB-D
    scenes", "multi-camera surround view")
mechanism_facets: What are the key mechanisms or frameworks? (e.g., "camera-space
    rotation augmentation", "SSM-based state space modeling")
assumption_facets: What are the key assumptions? (e.g., "calibrated pinhole camera",
     "no depth supervision")
Research Idea: {idea_text}

Please output ONLY a JSON object with keys "problem_facets", "setting_facets", "
    mechanism_facets", "assumption_facets". Each value should be a list of short
    phrases (3-7 phrases max per category).

JSON output:
```

B2-P4: High-Level Facets from Paper

Function: `extract_high_level_facets_from_paper()` **LLM role:** Light **Output:** JSON object with problem/setting/mechanism/assumption facets

```
You are a JSON generator. Do NOT call any tools or write/run any code. Think
    silently and then output the final answer as JSON only. Return raw JSON only,
    without backticks or any surrounding text.

CRITICAL: Output ONLY valid JSON. No markdown, no comments, no trailing commas, no
    text before or after.

Given the following paper info, please extract high-level structural facets.

Facet Types:
problem_facets: What specific problem/task is it solving?
setting_facets: What is the setting, modality, or protocol?
mechanism_facets: What are the key mechanisms or frameworks?
assumption_facets: What are the key assumptions?
Paper Info: {context}

Please output ONLY a JSON object with keys "problem_facets", "setting_facets", "
    mechanism_facets", "assumption_facets". Each value should be a list of short
    phrases.
```

```
JSON output:
```

B2-P5: Relevance Keyword Generation for Lexical Prefilter

Function: `generate_relevance_keywords_from_idea()` **LLM role:** Expert **Output:** JSON list of keywords/phrases

```
You are a research assistant helping to filter academic papers. We are looking for
    prior art for the following idea:
Topic: {topic}
Idea: {idea}
Key Facets: {json.dumps(facets)}

Please generate a list of {max_keywords} relevance keywords or phrases. These
    keywords will be used to score papers (title + abstract) to filter out
    OBVIOUSLY irrelevant ones (e.g., NLP, Metaverse, Materials Science).

Guidelines:
Include specific technical terms (e.g., "monocular 3d detection", "rotation
    augmentation").
Include broader domain cues (e.g., "3d vision", "point cloud", "lidar", "rgb-d", "
    autonomous driving").
Include variations (e.g., "3d object detection", "3d detection").
The goal is RECALL. If a paper contains these words, it might be relevant.
Output strictly valid JSON: ["keyword1", "keyword2", ...] Do not output markdown
    code blocks.
```

B2-P6: LLM Relevance Prefilter (Boolean)

Function: `llm_relevance_prefilter()` **LLM role:** Light **Output:** `{"is_relevant": true/false}`

```
You are an academic research assistant. Quickly determine if this paper might be
    relevant to the research topic below.

Research Topic: {topic}
Research Idea: {idea_description}

Paper Title: {paper.title}
Paper Abstract: {paper.abstract or "N/A"}

Is this paper potentially relevant to the research topic/idea above?
Only output valid JSON, no explanation: {"is_relevant": true} or {"is_relevant":
    false}
```

B2-P7: Abstract-Only Scoring + Paper High-Level Facets (Merged Call)

Function: `score_papers_with_abstract_only()` (merged) **LLM role:** Light **Output:** JSON object with overlap + extracted paper facets

```
You are a JSON generator. Do NOT call any tools or write/run any code. Think
    silently and then output the final answer as JSON only. Return raw JSON only,
    without backticks or any surrounding text.

Given the following research idea facets and a paper, do TWO tasks:
1) Rate how much this paper overlaps with the idea (scoring)
2) Extract the paper's own high-level facets

Idea Facets: {facets_str}
Paper: {paper_context}
```

```
Please output a SINGLE JSON object with ALL of the following fields:

"coverage_level": integer 0-3 where: 0 = no overlap 1 = adjacent work (same domain
    but different approach) 2 = partial overlap (covers some facets) 3 = high
    overlap (covers most/all facets, potential conflict)
"covered_facets": list of facet labels (e.g., ["F1", "F3"]) that this paper covers
"task_similarity": float 0.0-1.0 how similar the task/problem addressed is
"method_similarity": float 0.0-1.0 how similar the method/approach is
"notes": brief justification (1-2 sentences)
"high_level_facets": object with these keys (each is a list of short phrases):
"problem_facets": What specific problem/task is this paper solving?
"setting_facets": What is the setting, modality, or protocol?
"mechanism_facets": What are the key mechanisms or frameworks?
"assumption_facets": What are the key assumptions?

JSON output:
```

B2-P8: Abstract-Only Scoring (Fallback: Scoring Only)

Function: `score_papers_with_abstract_only()` (fallback) **LLM role:** Light **Output:** JSON object with overlap fields

```
You are a JSON generator. Do NOT call any tools or write/run any code. Think
    silently and then output the final answer as JSON only. Return raw JSON only,
    without backticks or any surrounding text.

Given the following research idea facets and a paper, rate how much this paper
    overlaps with the idea.

Idea Facets: {facets_str}
Paper: {paper_context}

Please output a JSON object with:
"coverage_level": integer 0-3 where: 0 = no overlap 1 = adjacent work (same domain
    but different approach) 2 = partial overlap (covers some facets) 3 = high
    overlap (covers most/all facets, potential conflict)
"covered_facets": list of facet labels (e.g., ["F1", "F3"]) that this paper covers
"task_similarity": float 0.0-1.0 how similar the task/problem addressed is
"method_similarity": float 0.0-1.0 how similar the method/approach is
"notes": brief justification (1-2 sentences)

JSON output:
```

A.3 Block 3: Full-Text Refinement Prompts

B3-P1: Full-Text Refinement (Coverage + Role)

Function: `refine_top_papers_with_fulltext()` **LLM role:** Light **Output:** `{"coverage_level_refined":0-3, "role":..., "notes":...}`

```
You are checking for potential prior art.

Research idea: {idea_description}

Key facets:
F1: {facets[0]}
F2: {facets[1]} ...

Paper:
Title: {paper.title}
Venue: {paper.venue or 'Unknown'}
Year: {paper.year or 'Unknown'}
Citations: {paper.citation_count or 0}
```

```
Abstract: {paper.abstract or 'N/A'}

Relevant snippets from the full text:
{snippets_text}

Based on ALL of the above, answer in JSON:
{ "coverage_level_refined": 0, "role": "main_contribution" | "key_technique" | "
    background" | "irrelevant", "notes": "short justification" }

Return RAW JSON only, no backticks.
```

A.4 Post-Processing Prompts (Method/Experiment Snippets and Key Paper Selection)

PP-P1: Paragraph Filtering and Bullet Summaries for Method/Experiment Snippets

Function: `_llm_filter_paragraphs()` **LLM role:** Light **Output:** JSON array with relevance + bullet + snippet

```
You are analyzing research paper paragraphs for relevance to a specific technical
    facet.

Facet being analyzed: "{facet_label}"

Paragraphs to evaluate:
Para 1 (paper=xxx, section=method): [Previous]: ... [Current]: ... [Next]: ...
Para 2 ...

For each paragraph, determine if it is RELEVANT to the facet and if so, generate a
    concise bullet summary.

Output a JSON array with one object per paragraph. Each object must have exactly
    these fields:
"para_num": the paragraph number (1, 2, 3, etc.)
"relevant": true or false
"bullet": a one-sentence summary (only if relevant, otherwise empty string)
"snippet": key 1-2 sentences from the original text (only if relevant, otherwise
    empty string)
"relevance_score": 0.0 to 1.0 (0 if not relevant)

Output ONLY the JSON array, no other text.
```

PP-P2: Key Paper Selection (Primary + Alternates per Bucket)

Function: `_select_key_papers_llm()` **LLM role:** Heavy **Output:** JSON object with `top_cited/latest/survey` selections

```
You are selecting key representative papers for a literature review on: "{topic}"

Available candidates by category:

TOP CITED:
[paper_id] Paper Title (2023, cited=150)
...

LATEST:
...

SURVEYS:
...

For each category, select:
1 PRIMARY paper (most representative/important)
0-2 ALTERNATES (complementary perspectives)
```

```
Criteria:
PRIMARY should be the most influential or representative
ALTERNATEs should provide different perspectives, not duplicate the primary
If a paper appears in multiple categories, prefer it as primary in the most relevant
     one

Output a JSON object with this exact structure:
{ "top_cited": {"primary": "paper_id_here", "alternates": ["id1", "id2"]},
  "latest": {"primary": "paper_id_here", "alternates": []},
  "survey": {"primary": "paper_id_here_or_null", "alternates": []} }

Output ONLY the JSON, no other text.
```

A.5 Synthesis Prompt Templates

SYN-T1: Full Report Prompt (Introduction + Related Work Report)

Template: `FULL_REPORT_PROMPT` **Output:** Markdown report

```
You are an expert researcher helping to write the Introduction and Related Work
    sections for a technical paper.

You are given a single JSON object called SYNTHESIS that has the following structure
    :

SYNTHESIS.metadata:
topic: the broad research area (e.g., "monocular 3D object detection")
idea_description: the specific research idea we want to check for prior art
facets: a list of fine-grained aspects F1, F2, ... of the idea
task: describes WHY this mapping was built (task_type, user_query, user_intent_notes
    , constraints)
audience: who the report is for (e.g., "expert_researcher")
usage_context: how the report will be used (e.g., ["paper_introduction", "
    related_work_section"])

SYNTHESIS.global_stats: high-level counts and coverage statistics
SYNTHESIS.retrieval_summary: query_seeds, sources, exploration_notes
SYNTHESIS.limitations: retrieval_coverage, notes
SYNTHESIS.buckets: high_overlap, partial_overlap, adjacent (paper_ids)

SYNTHESIS.papers: per-paper entries with:
paper_id, title, year, venue, citation_count
overlap_level, overlap_bucket, facet_labels
key_roles, intro_rw_summary, intro_rw_full, method_exp_snippets

Your task is:
Carefully read the JSON.
Use SYNTHESIS.metadata.task to understand what the user actually wants.
Use the summaries as the main signal, and treat the full Intro/RW and method/
    experiment snippets as supporting evidence, not as something to copy verbatim.

Write a human-readable Markdown report for a researcher who wants to know:
What is the overall landscape of this topic?
Which papers are most similar to our idea?
Which papers are related but different, and how?
Where are the clear gaps and opportunities?

OUTPUT FORMAT (Markdown)
1. Problem Setting and Research Idea
2. Domain Overview
3. Highly Similar Work (Potential Conflicts)
4. Related Work with Partial Overlap
5. Adjacent and Broader Directions
```

LLM Role	Typical Usage	Calls / Run	Examples
Expert	Knowledge recall, query design	low (1–3)	B1-P1, B1-P2, B1-P5
Heavy	Complex selection/refinement	low (1–2)	B1-P7, PP-P2
Light	High-frequency filtering/scoring	high (80–200+)	B2-P6, B2-P7, B3-P1

Table 5: Model tier usage in the Prior Art Check pipeline.

```
6. Gaps, Opportunities, and Positioning of Our Idea
7. Limitations of This Mapping

At the very top of your answer, include a one-paragraph "TL;DR" summarizing whether
    there seems to be any obvious high-overlap prior art and what the most
    important related directions are.
```

SYN-T2: Short Related Work Prompt

Template: `SHORT_RW_PROMPT` **Output:** Markdown "Related Work" section only

```
You are given a JSON object SYNTHESIS with metadata, facet definitions, and a list
    of prior papers related to a research idea.

Write ONLY a "Related Work" section (without Introduction) in Markdown, with:
- 2-3 paragraphs on the overall landscape of the topic.
- 2-3 paragraphs on the most similar work (from high_overlap and partial_overlap).
- 1 paragraph on adjacent directions.

You MUST:
- Mention papers with their title and year.
- Explain similarities and differences to the target idea.
- Avoid long verbatim quotes.
- Briefly mention how papers were retrieved using SYNTHESIS.retrieval_summary (1-2
    sentences).
- Do not repeat the JSON itself. Just write the final text.
```

A.6 LLM Role Usage Summary

Table 5 summarizes model tiers used by the pipeline and their typical call volume per run.